



เอกสารตัวก่อน Present — Nuclear Re:Mind

นิวเคลียร์เปลี่ยนความคิดโลก · NSC 2026 หมวดโปรแกรมส่งเสริมทักษะเพื่อการเรียนรู้ · โรงเรียนศึกษานารีวิทยา

หลักคิดที่สำคัญที่สุดของเอกสารนี้: กรรมการไม่ได้ทดสอบว่าคุณ "ท่องโค้ดได้" แต่ทดสอบว่าคุณ **เข้าใจระบบที่ตัวเองสร้าง**
+ ตัดสินใจออกแบบอย่างมีเหตุผล + แก้ปัญหาเป็น อ่านเอกสารนี้ให้เข้าใจ แล้วพูดด้วยภาษาของตัวเอง อย่าท่องเป็นนกแก้ว

ส่วนที่ 1 — Pitch 30 วินาที (ท่องให้ขึ้นใจ)

🔗 **พูดแบบนี้:** "Nuclear Re:Mind เป็นเกม City Builder แนว Simulation-Based Learning ที่แก้ปัญหา **Nuclear Phobia** — คนไทยส่วนใหญ่กลัวพลังงานนิวเคลียร์เพราะภาพจำจากภัยพิบัติ ทั้งที่มันเป็นพลังงานสะอาดที่สำคัญต่ออนาคต ผู้เล่นรับบท Dr. Auren Vasek วิศวกรนิวเคลียร์ที่ต้องฟื้นฟูเมืองร้าง Veltara และสร้างเตาปฏิกรณ์ฟิวชัน CORE TOWER ให้เสร็จก่อนทรัพยากรหมด ระหว่างเล่นผู้เล่นจะได้เรียนรู้ฟิสิกส์นิวเคลียร์จริง — Fission, Half-life, การกักขังรังสี — ผ่าน Learning Codex, Quiz และการตัดสินใจเชิงจริยธรรมในสถานการณ์วิกฤต เราเชื่อว่าการ 'เล่นแล้วเข้าใจ' เปลี่ยนทัศนคติได้ดีกว่าการอ่านตำรา"

ส่วนที่ 2 — สถาปัตยกรรมโปรแกรม (หัวใจของคำถามเชิงเทคนิค)

2.1 ภาพใหญ่: "ผู้จัดการหลายคน + ห้องสารบรรณกลาง"

โค้ดทั้งเกมแบ่งเป็น **Manager** (ผู้จัดการ) หลายตัว แต่ละตัวดูแลเรื่องเดียวของตัวเอง และห้ามคุยกันตรง ๆ — ทุกอย่างต้องส่ง "ประกาศ" ผ่านศูนย์กลางชื่อ `EventManager`

เปรียบเทียบ: เหมือนโรงเรียนที่มีหัวหน้าฝ่ายวิชาการ ฝ่ายการเงิน ฝ่ายอาคาร — เวลาเป็นเรื่องจะไม่เดินไปสั่งกันเอง แต่ประกาศผ่านเสียงตามสาย ใครสนใจเรื่องไหนก็ "สมัครรับฟัง" เรื่องนั้น ศัพท์ทางการเรียกว่า **Observer Pattern (Event-driven)**

ตัวอย่างจริงในเกม — ตอนวางอาคาร 1 หลัง เกิดอะไรขึ้น:

- `PlacementController` ตรวจสอบว่า พื้นว่าง + ทรัพยากรพอ → ประกาศ "มีตึกถูกวางแล้ว" (`OnBuildingPlaced`)
- `ResourceManager` ได้ยิน → หักค่าก่อสร้าง (เหล็ก + พลังงาน)
- `ConstructionController` ได้ยิน → เอาตึกเข้าคิวก่อสร้าง นับเวลา
- `BuildingVisualSpawner` ได้ยิน → วาดภาพตึกลงบนแมพ
- พอสร้างเสร็จ → ประกาศ "ก่อสร้างเสร็จ" → `ResourceManager` เริ่มให้ตึกนี้ผลิตทรัพยากรทุก tick

🔗 **พูดแบบนี้:** ถ้ากรรมการถาม "ทำไมไม่ให้มันเรียกกันตรง ๆ ง่ายกว่าไหม" ตอบว่า: "ตอนระบบเล็กมันง่ายกว่าจริงครับ แต่พอมี 15+ ระบบ ถ้าเรียกตรง ๆ ทุกระบบจะพันกันหมด แก้วเจ็ดเตี้ยฟังทั้งเกม แบบ Event ทำให้เราเพิ่มระบบใหม่ได้โดยไม่แตะโค้ดเก่าเลย เช่น ตอนเพิ่มระบบแจ้งเตือน (Toast) เราแค่ให้มัน 'สมัครฟัง' event ที่มีอยู่แล้ว ไม่ต้องแก้ Manager ตัวไหนเลย"

2.2 ค่าตัวเลขทั้งหมดอยู่ใน ScriptableObject ไม่ใช่ในโค้ด

ราคาตึก อัตราการผลิต ขนาดตึก ข้อความความรู้โนวเคลียร์ — ทั้งหมดเก็บเป็นไฟล์ข้อมูล (BuildingData , DilemmaData , CodexEntry) แยกจากโค้ด **เหตุผล:** ปรับสมดุลเกม (balancing) ได้โดยไม่ต้องแก้โปรแกรม คนในทีมที่ไม่เขียนโค้ดก็ปรับค่าได้ใน Unity Editor

2.3 ตาราง Manager ทุกตัว — "ใครทำอะไร" (ตอบได้ทุกตัว = มั่นใจได้เลย)

ไฟล์/ระบบ	หน้าที่ (ภาษาคน)
GameManager	คุมสถานะเกม: กำลังเล่น / ชนะ / แพ้ และความเร็วเวลา (หยุด, x1, x2...)
TimeManager	นาฬิกาของเกม — นับ tick และวัน (Day 1 → 30) แล้วประกาศให้ระบบอื่นทำงานตามรอบ
EventManager	ห้องสารบรรณกลาง — รวม event ทุกชนิด ทุกระบบสื่อสารผ่านตัวนี้
ResourceManager	คลังทรัพยากร (พลังงาน น้ำ อาหาร เหล็ก ดินที่เตรียม) — บวกการผลิต หักการบริโภค ทุก tick
PopulationManager	ประชากร 3 อาชีพ (แรงงาน/วิศวกร/แพทย์) + ค่าความหวัง (Hope) + การเสียชีวิตเมื่อวิกฤต
BuildingRegistry	สมุดทะเบียนตึกที่วางแล้ว: ตึกไหนอยู่ช่องไหน เลเวลเท่าไร + จัดการอัปเกรด
CoreTowerManager	ความคืบหน้า CORE TOWER (เริ่ม 30%) — ถึง 100% = ชนะเกม
GridManager	กริด Isometric — แปลงพิกัดช่องตาราง ↔ ตำแหน่งบนจอ, จำว่าช่องไหนถูกจอง
PlacementController	โหมดวางตึก: ghost ตามเมาส์ เชี่ยว=วางได้แดง=ไม่ได้ (ที่ไม่วาง/จองไม่พอ/ตึก unique ซ้ำ)
ConstructionController	คิวก่อสร้าง — ตึกใช้เวลาสร้าง เสร็จแล้วถึงเริ่มผลิต
DemolitionController	โหมดทุบตึก คืนทรัพยากรบางส่วน (ห้ามทุบ CORE TOWER — กันเกม soft-lock)
DilemmaManager	เหตุการณ์วิกฤต + ทางเลือกเชิงจริยธรรม 3 ทาง แต่ละทางมีผลต่อทรัพยากร/ความหวัง/ชีวิตคน
QuizManager + Popup	แบบทดสอบความรู้โนวเคลียร์ — สุ่มสลับลำดับตัวเลือกทุกครั้ง (กันจำข้อ)
CodexManager	สารานุกรมความรู้ในเกม ปลดล็อกตามความคืบหน้า
SaveManager	เซฟ/โหลดเกมเป็นไฟล์ JSON (F5 เซฟ / F9 โหลด)
CameraController	กล้องแบบ city builder: เลื่อนนุ่มมีแรงเฉื่อย, ลากด้วยคลิกกลาง, ซูมเข้าหาเมาส์
TutorialManager	สอนเล่นวันแรก ทีละขั้น

ส่วนที่ 3 — ระบบเกมเพลย์หลัก อธิบายแบบเข้าใจง่าย

3.1 Loop หลักของเกม (วงจรที่ผู้เล่นทำซ้ำ)

วางแผนอาคาร → รอก่อสร้าง → ผลิตทรัพยากร → เอาทรัพยากรไปสร้าง/อัปเกรดเพิ่ม + ป้อน CORE TOWER → เจอ
วิกฤต/ตัดสินใจ → CORE TOWER 100% ก่อนทรัพยากรหมด = ชนะ

3.2 กริด Isometric (ถ้าโดนถามเรื่องคณิตศาสตร์)

แมพเป็นตารางมุมเฉียง 2D (รูปขนมเปียกปูน) การแปลงพิกัดช่อง (col, row) → ตำแหน่งบนโลก:

$$x = (col - row) \times \text{ครึ่งความกว้าง tile} \quad | \quad y = (col + row) \times \text{ครึ่งความสูง tile}$$

แปลว่า เดินไปทาง col ภาพจะเลื่อน "ขวา-ขึ้น" เดินไปทาง row จะเลื่อน "ซ้าย-ขึ้น" — บวกกันเลยได้มุมเฉียงแบบ isometric สูตรนี้มี
unit test คมไว้ ห้ามใครแก้

3.3 เศรษฐกิจทรัพยากร

- 5 ทรัพยากรหลัก: พลังงาน ⚡, น้ำ 💧, อาหาร 🍗, เหล็ก ⚙️ (ไว้ก่อสร้าง), ดินที่เตรียม (เชื้อเพลิงฟิวชัน)
- ตักผลิต: โรงไฟฟ้า→พลังงาน, โรงน้ำ→น้ำ, ฟาร์ม→อาหาร, เหมือง→เหล็ก, ท่ออยู่อาศัย→เพิ่มเขตแดนประชากร, ห้องแล็บ→วิจัย/ฝึก
อาชีพ
- อัปเกรดตึก (ปุ่ม U): เลเวล 1→2→3 ผลิตเพิ่ม $\times 1 \rightarrow \times 3 \rightarrow \times 7.5$ (ยิ่งอัปยิ่งคุ้ม แต่ค่าอัปแพงขึ้นตามเลเวล)
- ทุกตึกมี upkeep (ค่าบำรุง) — สร้างเยอะเกินโดยไม่มีฐานผลิต = เจ๊ง เป็นบทเรียน Systems Thinking ตรง ๆ

3.4 ประชากร + ความหวัง (Hope) — ระบบ "มนุษย์" ของเกม

- ประชากร 3 อาชีพ: แรงงาน (ทำงานในตึก), วิศวกร (จำเป็นต่อ CORE TOWER), แพทย์ (ฟื้นฟูความหวัง) — ฝึกอาชีพได้ที่ห้อง
แล็บ
- Hope (ความหวัง): อาหารขาด $-10/\text{วัน}$ · มีแพทย์และไม่ขาดแคลน $+2/\text{วัน}$ · มีคนเสียชีวิต $-5/\text{คน}$ · แก้วิกฤตสำเร็จ $+5$
- วิกฤตบางทางเลือก "ประหยัดทรัพยากรแต่มีคนตาย" — นี่คือจุดที่เกมสอน Ethical Decision Making ตามวัตถุประสงค์โครงการ

3.5 เหตุการณ์วิกฤต (Dilemma/Crisis) — เชื่อมความรู้นิวเคลียร์กับการตัดสินใจ

วิกฤต	เงื่อนไขเกิด (trigger)	สอนอะไร
ปลาสมาร้อนเกิน	ความร้อนเตา > 80% หรือ ความคืบหน้าเตา > 30%	Coolant System / การป้องกัน meltdown
โรคระบาด	ถึงวันที่ 20	การแพทย์นิวเคลียร์ (ฉายรังสีฆ่าเชื้อ) + ทรัพยากรสาธารณสุข
อาหารล้นคลัง	อาหาร > 500	การกนอมอาหารด้วยรังสี (Food Irradiation)

ระบบ trigger เขียนเป็นเงื่อนไขแบบข้อความ เช่น `heat_above_80|q_above_0.3` (| = หรือ) — ทีมเพิ่มวิกฤตใหม่ได้โดยสร้างไฟล์
ข้อมูล ไม่ต้องแก้โค้ด

3.6 ระบบการเรียนรู้ (จุดขายของหมวดนี้!)

- **Tooltip 3 ชั้น:** ชีตักจะเห็น ชื่อ → หน้าที่ในเกม → **ความรู้науเคลียร์จริงเบื้องหลัง** เช่น เหมือนอธิบายการสกัด Lithium-6 เพื่อผลิต Tritium เชื้อเพลิงฟิวชัน
- **Learning Codex:** สารานุกรมปลดล็อกตามความคืบหน้า อ่านย้อนหลังได้ตลอด
- **Quiz:** แต่งถามเมื่อถึงจังหวะสำคัญ (เช่น ได้ดาวที่เริ่มครั้งแรก) — **ตัวเลือกสลับลำดับทุกครั้ง** ด้วยอัลกอริทึม Fisher-Yates กันผู้เล่นจำตำแหน่งข้อถูก

3.7 ระบบเซฟ

เก็บทุกอย่างเป็น JSON: ทรัพยากร ประชากร ตึกทุกหลัง+ตำแหน่ง ความคืบหน้าเตา — กติกาสำคัญของทีมคือ **ฟิวด์ใหม่ใน SaveData ต้องมีค่า default เสมอ** เพื่อให้เซฟเวอร์ชันเก่ายังโหลดได้ (backward compatibility)

ส่วนที่ 4 — เรื่องเล่า Debug (อาวุธลับตอนถูกถาม "เจอปัญหาอะไรบ้าง")

กรรมการชอบมากเวลาผู้พัฒนาเล่าปัญหาจริงพร้อมวิธีไล่หาสาเหตุ — เตรียม 2 เรื่องนี้ไว้เล่า

เรื่องที่ 1: "ตึกทั้งเมืองไม่ผลิตทรัพยากรเลย" (บั๊กที่ยากสุด)

💡 **พุดแบบนี้:** "อาการคือวางตึกเสร็จแล้วแต่ทรัพยากรไม่ขึ้นเลยสักหลัง เราไล่จาก event ที่ละชั้น พบว่ามีระบบโครงข่ายไฟฟ้าเก่าที่เช็คชื่อตึกเป็นภาษาอังกฤษ ('PowerPlant') แต่ข้อมูลตึกจริงของเราตั้งชื่อภาษาไทย ('โรงไฟฟ้า') มันเลยหาโรงไฟฟ้าไม่เจอ ระบบตีความว่าทั้งเมืองไม่มีไฟ แล้วบล็อกการผลิตทุกตึก บทเรียนคือการเทียบข้อมูลด้วย string ตรง ๆ เพราะบางมาก และระบบที่ไม่อยู่ในเอกสารออกแบบ (GDD) ไม่ควรมี side effect ใหญ่ขนาดนั้น — สุดท้ายเราตัดระบบนั้นออกให้ตรงกับ GDD"

เรื่องที่ 2: "หน้า Main Menu กดอะไรไม่ได้เลย"

💡 **พุดแบบนี้:** "ปุ่มแสดงผลปกติแต่คลิกไม่ติด สาเหตุคือ Unity ต้องมี EventSystem ในซีนถึงจะรับคลิกได้ ตอนสร้างซีนเมนูด้วยสคริปต์อัตโนมัติ โค้ดเช็คว่ามี EventSystem หรือยัง' แต่มันไปเจอของซีนเกมที่เปิดค้างอยู่ เลยคิดว่ามีแล้ว ไม่สร้างให้ พอเปิดซีนเมนูเดียว ๆ จึงไม่มีตัวรับ input เลย — แก้โดยบังคับสร้างในซีนใหม่เสมอ"

เรื่องเสริม: ป้องกันบั๊กด้วย Automated Test

เรามี **EditMode Test** ประมาณ 160 เคส คมตรระสำคัญ เช่น สูตรแปลงพิกัด isometric, กติกาห้ามวางตึกซ้อน, ห้ามวางเมื่อทรัพยากรไม่พอ, การคำนวณ Hope, การสุ่มสลับตัวเลือก Quiz — รันซ้ำได้ทุกครั้งก่อน commit ทำให้แก้บั๊กใหม่แล้วมั่นใจว่าของเก่าไม่พัง (regression testing)

ส่วนที่ 5 — คำถามที่น่าจะโดนถาม + แนวตอบ

Q1: โครงการนี้แก้ปัญหาอะไร ต่างจากการดูสารคดี/อ่านหนังสือยังไง?

สารคดีให้ "ความรู้" แต่เกมให้ "ประสบการณ์" — ผู้เล่นต้องบริหารเอาปฏิกิริยาเอง เจอผลของการตัดสินใจเอง เช่น ถ้าละเลยระบบหล่อเย็นจะเจอวิกฤตพลาสมา นี่คือการ Learning by Doing ที่สร้าง Systems Thinking ได้จริง และเป้าหมายลึกกว่านั้นคือเปลี่ยนทัศนคติ (Nuclear Phobia) ซึ่งการบรรยายทำได้ยาก

Q2: วัดผลการเรียนรู้อย่างไรว่าผู้เล่น "ได้ความรู้จริง"?

ในเกมมี Quiz ท้ายช่วงสำคัญเก็บคะแนนได้ และแผนการทดสอบคือทำ Pre-test / Post-test กับกลุ่มตัวอย่างนักเรียน เทียบคะแนนความรู้และแบบวัดทัศนคติต่อพลังงานนิวเคลียร์ก่อน-หลังเล่น

Q3: ทำไมเลือก Unity + C#?

Unity เหมาะกับเกม 2D isometric บน PC, มีระบบ ScriptableObject ให้แยกข้อมูลออกจากโค้ด, มี Test Framework ในตัว, เอกสารและชุมชนใหญ่ทำให้ทีมนักเรียนเรียนรู้ได้เร็ว และ build ลง Windows ได้ทันที

Q4: สถาปัตยกรรมโปรแกรมเป็นแบบไหน ทำไม?

Singleton Manager + Event-driven (Observer Pattern) — ทุกระบบสื่อสารผ่าน EventManager กลาง ข้อดีคือระบบไม่พันกัน (loose coupling) เพิ่มฟีเจอร์ใหม่ได้โดยไม่แตะโค้ดเก่า และทดสอบแต่ละระบบแยกกันได้ (ดูรายละเอียดส่วนที่ 2 — ตอบข้อนี้ให้คล่องที่สุด)

Q5: เนื้อหาวิชาเคมีถูกต้องไหม อ้างอิงจากไหน?

อิงหลักฟิสิกส์มาตรฐาน: Fission/Chain Reaction, Half-life, การกำบังรังสี (เวลา-ระยะทาง-เครื่องกำบัง), พิวชัน D-T และการผลิต Tritium จาก Lithium-6, การประยุกต์ (การแพทย์/เกษตร/ถนอมอาหาร) โดยมีครูที่ปรึกษาตรวจเนื้อหา — **[เติมชื่อแหล่งอ้างอิงจริงของทีม เช่น ตำรา/เว็บ IAEA ก่อนวันจริง]** จุดที่ต้องยอมรับตรง ๆ ถ้าถูกใจ: ตัวเลขในเกมถูก "ปรับสเกลเพื่อความสนุก" แต่หลักการถูกต้อง

Q6: ส่วนไหนพัฒนายากที่สุด?

เล่าเรื่อง Debug ที่ 1 (ติกไม่ผลิตทรัพยากร) จากส่วนที่ 4 — เป็นคำตอบที่แสดงทักษะไล่ปัญหาเชิงระบบได้ดีที่สุด

Q7: ทดสอบโปรแกรмыังไง?

2 ชั้น: (1) Automated EditMode Test ~160 เคสคุมตรรกะเกม รันก่อน commit ทุกครั้ง (2) Playtest เล่นจริงครบ 30 วันในเกม เพื่อทดสอบดุลและ UX แล้วเก็บ feedback มาปรับ

Q8: ใช้ AI ช่วยพัฒนาไหม? (ตอบตามความจริง อย่าโกหกเด็ดขาด)

"ใช้ครับ เราใช้ AI (Claude) เป็นผู้ช่วยเขียนโค้ดตามดีไซน์ที่ทีมกำหนด เหมือนที่นักพัฒนามืออาชีพใช้กัน โดยทีมเป็นคนออกแบบระบบเกม กติกา ค่าสมดุล และเนื้อหาความรู้ทั้งหมดใน GDD, ส่งงาน-ตรวจรับทีละฟีเจอร์, ทดสอบเล่นจริงและรายงานบั๊กให้แก ทีมเข้าใจสถาปัตยกรรมและอธิบายทุกระบบได้ (แล้วอธิบายส่วนที่ 2 ให้ฟังทันที)"

สำคัญ: ตรวจกติกา NSC ปีนี้อธิบายการใช้ AI ให้ชัดก่อนวันจริง และเตรียมอธิบายระบบใดก็ได้ที่กรรมการชี้

Q9: แผนต่อยอด?

เติมอาคารตามแผน (Agri Dome, โรงพยาบาล), ระบบเสียง, ปรับสมดุลจาก playtest จริง, วัดผลกับกลุ่มนักเรียนจริง, และเผยแพร่เป็นสื่อการสอนให้โรงเรียนอื่นใช้ฟรี

Q10: ถ้ามีเวลา/ทรัพยากรมากกว่านี้ จะทำอะไรเพิ่ม?

โหมดหลายระดับความยาก, Dashboard วิเคราะห์การตัดสินใจของผู้เล่นให้ครูใช้ประเมิน, แปลอังกฤษ, และ port ลงแพลตฟอร์มอื่น

ส่วนที่ 6 — สคริปต์เดโม 3 นาที

เตรียมก่อนขึ้นเวที: (1) เปิดเกมค้างไว้ที่ Main Menu (2) มีไฟล์เซฟกลางเกมในเมืองสวย ทรัพยากรเดินแล้ว เพื่อไว้กด F9 โหลด (3) ซ้อมตามสคริปต์นี้จนพูดได้โดยไม่ดูกระดาษ อย่างน้อย 5 รอบ (4) มีวิดีโอเดโมสำรองในเครื่อง เผื่อเกม crash หน้านาน

เวลา	ทำอะไรบนจอ	พูดอะไร
0:00–0:20	หน้า Main Menu → กดเริ่มเกม	Pitch 30 วิ ฉบับย่อ: เกมแก้ Nuclear Phobia ด้วย Simulation-Based Learning รับทวิศกรฟื้นฟูเมือง สร้างเตาฟิวชันให้เสร็จ
0:20–0:50	แพนกล้อง WASD รอบเมือง, ชุมเข้า CORE TOWER กลางแมพ, กด F1 โชว์แผนที่ลัดแนวหนึ่ง	"นี่คือเมือง Veltara — CORE TOWER เตาปฏิกรณ์ฟิวชันที่สร้างค้างไว้ 30% ภารกิจคือทำให้ถึง 100% ก่อนทรัพยากรหมด"
0:50–1:30	กดเลข 1 เลือกโรงไฟฟ้า → ghost เชียว/แดง → วาง → ชี้ให้ดูคิวก่อสร้าง → ชี้ HUD ทรัพยากรที่ขยับ	"วางอาคารต้องคิดเหมือนวิศวกรจริง — พื้นที่ ทรัพยากร ค่าบำรุง ถ้าวางไม่ได้ระบบจะขึ้นสีแดงบอกเหตุผล ทุกดีกผลิตทรัพยากรเข้าระบบเศรษฐกิจแบบเรียลไทม์"
1:30–2:00	ชี้เมาส์ค้างที่ตึก → โชว์ Tooltip 3 ชั้น → เปิด Codex สั้น ๆ	"จุดขายด้านการเรียนรู้: ทุกอาคารมีความรู้นิวเคลียร์จริงเบื้องหลัง เช่น เหมือนนี้สกัด Lithium-6 ไปผลิต Tritium เชื้อเพลิงฟิวชัน — เก็บอ่านย้อนหลังได้ใน Learning Codex"
2:00–2:35	โหลดเซฟที่เตรียมไว้ (F9) → โชว์ Quiz หรือวิกฤต + ตัวเลือก 3 ทาง	"เมื่อถึงจุดสำคัญจะมี Quiz และวิกฤตเชิงจริยธรรม — บางทางเลือกประหยัดทรัพยากรแต่แลกด้วยชีวิตคน ความหวังของเมืองจะลดลง ผู้เล่นได้ฝึก Ethical Decision Making จริง ๆ"
2:35–3:00	ชี้แถบ Hope / ประชากร / ความคืบหน้าเตา แล้วหันหน้าหากรรมการ	"ทุกระบบ—พลังงาน อาหาร คน ความหวัง—เชื่อมโยงกันหมด นี่คือ Systems Thinking ที่เราอยากให้ผู้เล่นได้ และเปลี่ยนความกลัวนิวเคลียร์เป็นความเข้าใจ ขอขอบคุณครับ"

ส่วนที่ 7 — ศัพท์เทคนิคที่ควรพูดได้สิ้น

ศัพท์	ความหมายสั้น ๆ (ไว้ตอบเวลาโดนถามแทรก)
Singleton	คลาสที่มี instance เดียวทั้งเกม เข้าถึงได้จากทุกที่ — ใช้กับ Manager ทุกตัว
Observer / Event-driven	ระบบ "ประกาศ-สมัครรับฟัง" — คนประกาศไม่ต้องรู้ว่าใครฟังอยู่
ScriptableObject	ไฟล์ข้อมูลของ Unity แยกค่าตัวเลข/เนื้อหาออกจากโค้ด
Loose coupling	ระบบพึ่งพากันน้อย → แก่จุดหนึ่งไม่กระทบจุดอื่น
Unit test / Regression	โค้ดทดสอบโค้ด รันอัตโนมัติ — กันของเก่าพังตอนเพิ่มของใหม่
SmoothDamp	ฟังก์ชันเคลื่อนค่าเข้าหาเป้าหมายแบบนุ่มนวล ใช้ทำกล้องเลื่อน/ซูมสั้น ๆ
Fisher-Yates	อัลกอริทึมสับลำดับแบบสุ่มธรรม ใช้สลับตัวเลือก Quiz
JSON	รูปแบบข้อความเก็บข้อมูลมีโครงสร้าง ใช้กับระบบเซฟ

ส่วนที่ 8 — เช็กลิสต์ก่อนวันจริง

- ☐ ซ้อมเดโม 3 นาทีอย่างน้อย 5 รอบ (จับเวลาจริง)
- ☐ ทุกคนในทีมตอบ Q1–Q8 ได้ด้วยคำพูดตัวเอง — ซ้อมโดยให้เพื่อน/ครูสุ่มถาม
- ☐ เตรียมไฟล์เซฟ 2 จุด: ต้นเกม + กลางเกม (เมืองสวย มีวิกฤตใกล้เค้น)
- ☐ อัปเดตโอเดโมสำรอง เก็บทั้งในเครื่องและแฟลชไดรฟ์
- ☐ เต็มแหล่งอ้างอิงเนื้อหาในเคสรีจริงในคำตอบ Q5
- ☐ ตรวจกติกา NSC เรื่องการใช้ AI + เตรียมคำตอบ Q8 ให้ตรงกติกา
- ☐ ทดสอบเกมบนเครื่อง/จอโปรเจกเตอร์ที่จะใช้จริง (ความละเอียดจอต่างกัน UI อาจเพี้ยน)
- ☐ ชาร์จโน้ตบุ๊ก + พกสายชาร์จ + ปิด notification เครื่องก่อนขึ้นเวที

ประโยคปิดท้ายที่ใช้ได้เสมอเวลาตอบไม่ได้: "ขออนุญาตตอบตามที่เข้าใจนะครับ ถ้าคลาดเคลื่อนยินดีรับคำแนะนำครับ" — ดีกว่ามั่วแล้วโดนจับได้ 100 เท่า กรรมการให้คะแนนความซื่อสัตย์และ growth mindset